

Painel de Medicação

CRUD reativo, do zero, em par — o mesmo padrão da Semana 01, construído com as suas mãos

90min

no laboratório

Dupla

mesma da sala

Chat

IA permitida

0

vibe / agente



Uma atividade-espelho, não uma semana nova

Mesma arquitetura, mesmo ritmo, mesmos conceitos da Semana 01 — replicados do zero, em domínio novo, contra o relógio.

O QUE MUDA

O domínio e a pressão de tempo

Em vez de Patient/Encounter, hoje é MedicationOrder. Em vez de duas aulas de 2h, hoje são 90 minutos, sozinhos, com IA só em chat.

Se você entendeu de verdade na Semana 01, hoje vai mais rápido — não porque decorou código, mas porque reconhece o padrão.

O QUE NÃO MUDA

A arquitetura

Backend: Express + TypeScript + SQLite, SQL parametrizado, sem ORM, sem camadas.

Frontend: a mesma separação estado → renderização → comunicação. api.js é a única porta para fetch. Sempre.

Ao final dos 90 minutos: uma API com 4 operações e um frontend reativo, construídos por vocês — não copiados, entendidos.

Lembrando o ciclo da Semana 01



“O estado muda. A tela é consequência.” Hoje vocês vão escrever cada uma dessas cinco caixas de novo, para MedicationOrder — sem esqueleto de resposta pronta.

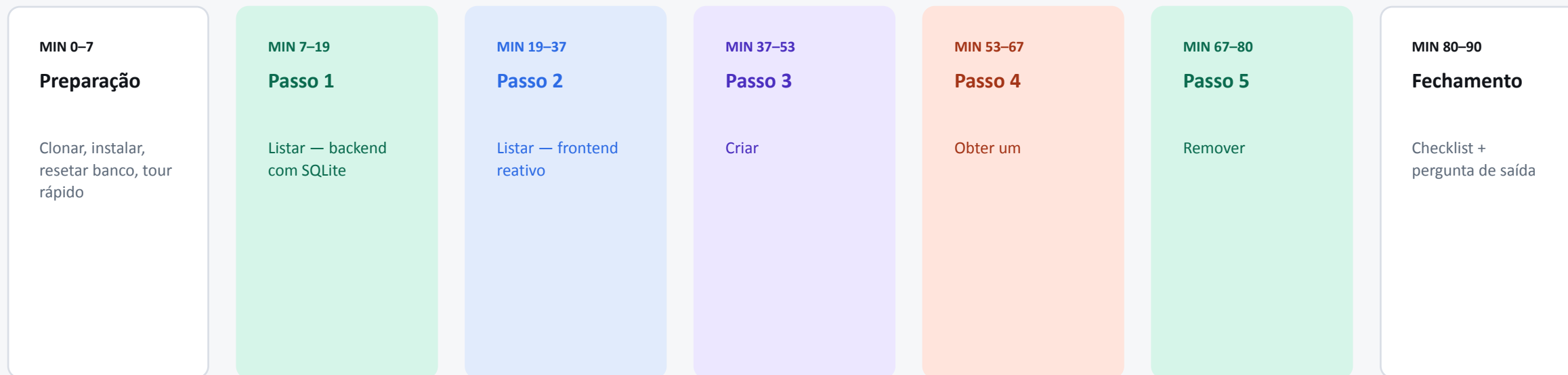
UM LEMBRETE TÉCNICO

Nenhum fetch fora de api.js. Nenhuma decisão em render.js.

Se você está em dúvida sobre onde uma linha de código deveria morar, essas duas regras resolvem 90% das dúvidas.

90 minutos, construção vertical

Cada operação inteira — backend e frontend — antes de passar para a próxima. Algo novo funciona a cada 15 minutos.



Se sua dupla estiver muito atrasada num passo, chame o professor — não vale a pena perder os passos seguintes travados em um só.

A regra de IA desta atividade

Não-negociável. Vale para as duas pessoas da dupla.

● VOCÊ PODE

IA como apoio pontual

- Perguntar o que um erro significa
- Pedir explicação de UMA linha por vez
- Pedir o rascunho de UMA função por vez
- Pedir revisão de um trecho que você mesmo escreveu

● VOCÊ NÃO PODE

IA pensando no seu lugar

- Colar o passo inteiro do roteiro e pedir o código pronto
- Usar modo agente (escreve o arquivo sozinha)
- Aceitar um trecho sem ler e sem saber explicar
- Deixar a IA decidir a arquitetura

Teste rápido: se você não consegue explicar uma linha para sua dupla, ela não devia estar no seu código ainda.

Chat, sempre. Agente, nunca.

PERMITIDAS NESTA ATIVIDADE

Conversa normal

- ChatGPT, Gemini ou Claude, em modo chat
- Você copia, cola, lê, adapta — sempre com as próprias mãos

PROIBIDAS NESTA ATIVIDADE

Execução automática

- Copilot em modo agente / Workspace
- Cursor Agent, Composer com auto-apply
- Claude Code ou qualquer CLI autônoma
- Qualquer extensão que escreva o arquivo sozinha

LEMBRETE DO PROTOCOLO (C·P·T·R·F·A)

Continua valendo, versão curta

Contexto (a stack e o que já existe) + Restrições (“sem ORM, sem libs extras, SQL parametrizado, JS puro”) são as duas partes que mais evitam resposta fora do escopo. Se a IA sugerir Prisma, React ou body-parser, ela respondeu a outra pergunta — não a sua.

Controle de medicação durante uma internação

Marginal ao Mini-Prontuário — não usa Patient nem Encounter — mas no mesmo universo de saúde.

A ANALOGIA

O quadro ao lado do leito

Todo leito de hospital tem um quadro com a medicação do paciente: o quê, quanto, por qual via, a que horas.

É um registro simples, de acesso rápido, sem burocracia — exatamente o que vamos construir: uma tela, um formulário, uma lista.

SIGNIFICADO CLÍNICO DO CRUD

“Remover” é suspender

Criar uma prescrição = médico prescreveu.

Listar = ver o quadro inteiro.

Obter uma = abrir o detalhe de uma prescrição específica.

Remover = suspender — uma ação real, não exclusão arbitrária.

Entender por que existe cada operação ajuda tanto quanto saber como implementá-la.

MedicationOrder

Repare: o mesmo formato de Encounter (Atividade 01). Reconhecer o padrão é o ponto — não decorar campos novos.

CAMPO	TIPO	OBSERVAÇÃO
id	número	gerado pelo servidor
patientName	texto	obrigatório
medicationName	texto	obrigatório
dosage	texto	obrigatório — ex: “500mg”
route	texto	obrigatório — Oral / Intravenosa / Intramuscular
scheduledAt	texto (ISO)	obrigatório — AAAA-MM-DDTHH:MM
notes	texto	opcional

As quatro rotas que vocês vão construir

CONTRATO DA API

```
GET      /api/medications      -> 200 [ MedicationOrder ]
POST     /api/medications      -> 201 MedicationOrder | 400
GET      /api/medications/:id  -> 200 MedicationOrder | 404
DELETE   /api/medications/:id  -> 204 (sem corpo)      | 404
```

SEM PUT NEM PATCH

Só C-R-R-D, de propósito

Menos uma operação = mais tempo de profundidade nas outras quatro.
Atualização fica para outra atividade.

STATUS NOVO: 204

“Removi, não tenho nada a devolver”

Soma-se aos cinco já vistos (200/201/400/404/409). Repertório de status cresce naturalmente, sem bloco de exposição.

O que já existe — não é boilerplate disfarçado

`npm install && npm run db:reset && npm run dev` — e o resto deste slide já está de pé.

Banco pronto

`schema.sql`, `seed.sql`, `database.ts` — 3 prescrições fictícias já no banco

Hello World no backend

`src/server.ts` só tem GET `/api/health` e o servidor estático

HTML e CSS prontos

formulário, lista e painel de detalhe existem na tela — vazios de dado

JS em esqueleto

`api.js`, `state.js`, `render.js`, `app.js` têm PASSO N marcando onde escrever

O ponto de partida real

Abra <http://localhost:3000> agora. É isto que você deve ver:

NA TELA

Formulário e lista vazia

O formulário existe e não faz nada ainda. A lista mostra “Carregando...” para sempre — porque a rota que ela chama não existe no servidor.

NO TERMINAL

GET /api/medications = 404

Teste no navegador ou no requests.http. É esperado — essa rota é o Passo 1, e vocês ainda não chegaram lá.

“Quase do zero” significa isto: a tela e a estrutura já existem para vocês não travarem no início, mas nenhuma lógica de CRUD foi escrita ainda.

7–19 MIN

Passo 1 — Listar (backend)

GET /api/medications: SQL parametrizado + tradução snake_case → camelCase.



O que escrever

ONDE ENTRA (PROCURE O COMENTÁRIO PASSO 1)

```
app.get('/api/medications', (_request, response) => {  
  const rows = db  
    .prepare(  
      `SELECT id, patient_name, medication_name, dosage,  
        route, scheduled_at, notes  
        FROM medication_orders  
        ORDER BY scheduled_at ASC`  
    )  
    .all();  
  
  response.status(200).json(rows.map(toMedicationJson));  
});
```

FALTA UMA PEÇA

toMedicationJson

Uma função que traduz patient_name → patientName.
Mesmo padrão de toPatientJson na Semana 01. Escreva-a antes da rota.

DICA DE ARRANQUE

Teste sem o frontend

Abra localhost:3000/api/medications direto no navegador.
Deve aparecer um JSON com 3 registros.

Antes de seguir para o Passo 2

- 1 GET /api/medications responde 200
- 2 O corpo é um array com 3 objetos
- 3 Os campos vêm em camelCase (patientName, não patient_name)
- 4 O SQL usa .prepare() — sem concatenação de string

Os quatro itens acima confirmados? Ótimo — o Passo 2 vai fazer essa mesma lista aparecer na tela.

19–37 MIN

Passo 2 — Listar (frontend reativo)

O maior passo do dia: reconstruir `api.js`, `state.js`, `render.js` e `app.js` do zero.



listMedications()

A ÚNICA FUNÇÃO DE REDE DESTA ETAPA

```
export async function listMedications() {  
  const response = await fetch(MEDICATIONS_URL);  
  
  if (!response.ok) {  
    throw new Error(  
      `Não foi possível carregar (HTTP ${response.status})`  
    );  
  }  
  
  return response.json();  
}
```

LEMBRETE

fetch não lança erro em 404/500

response.ok é obrigatório aqui. Sem ele, um erro do servidor vira sucesso silencioso no frontend.

REGRA DE OURO

Nenhum outro arquivo chama fetch

Se você digitar fetch fora deste arquivo hoje, pare e pergunte por quê.

setMedications()

A AÇÃO QUE GUARDA O RESULTADO

```
export function setMedications(medications) {  
  state.medications = medications;  
  state.isLoading = false;  
  state.errorMessage = null;  
  notify();  
}
```

TESTE MENTAL

Este arquivo conhece o DOM?

Não deveria. Se você escrever `document` aqui, algo saiu do lugar — igual na Semana 01.

POR QUE NOTIFY()

Ninguém mais precisa lembrar

Sem essa chamada, quem alterou o estado teria que lembrar de mandar redesenhar. Uma esquecida, e a tela dessincroniza.

renderMedicationList()

O DESENHISTA — BURRO DE PROPÓSITO

```
export function renderMedicationList(medications, container) {  
  if (medications.length === 0) {  
    container.innerHTML = emptyStateTemplate();  
    return;  
  }  
  
  container.innerHTML = medications  
    .map(medicationCardTemplate)  
    .join('');  
}
```

JÁ PRONTO PARA VOCÊ

medicationCardTemplate já existe

Está no mesmo arquivo, acima do comentário PASSO 2.
Você só precisa chamá-la — não escrevê-la de novo.

NENHUMA DECISÃO AQUI

render.js não filtra, não ordena

Se sobrou lógica de negócio nesta função, ela deveria estar
no state.js.

Ligando tudo: a carga inicial

O MAESTRO CHAMANDO PELA PRIMEIRA VEZ

```
async function start() {  
  renderApp(getState());  
  
  try {  
    const medications = await listMedications();  
    setMedications(medications);  
  } catch (error) {  
    setError(error.message);  
  }  
}  
  
start();
```

ORDEM IMPORTA

renderApp antes do fetch

A tela mostra “carregando” imediatamente, sem esperar a rede — é por isso que o primeiro render acontece antes do await.

ONDE CHAMAR RENDERMEDICATIONLIST

Dentro de renderApp

Já existe um comentário “// PASSO 2” marcando a linha exata dentro da função renderApp.

Antes de seguir para o Passo 3

- 1 Recarregar a página mostra os 3 cartões, vindos da API
- 2 “Carregando...” aparece brevemente antes da lista
- 3 O contador de prescrições está correto
- 4 Nenhum dado está “chumbado” direto no HTML

Este é o passo mais longo do dia. Se sua dupla chegou até aqui, o resto é repetir o mesmo padrão três vezes.

37–53 MIN

Passo 3 — Criar

POST /api/medications: validação no backend, formulário reativo no frontend.



POST /api/medications

VALIDAR ANTES DE INSERIR

```
app.post('/api/medications', (request, response) => {  
  const problem = validateMedicationInput(request.body);  
  if (problem) {  
    response.status(400).json({ error: problem });  
    return;  
  }  
  
  const { patientName, medicationName, dosage, route,  
    scheduledAt, notes } = request.body;  
  
  const result = db.prepare(  
    `INSERT INTO medication_orders  
      (patient_name, medication_name, dosage, route,  
       scheduled_at, notes)  
      VALUES (?, ?, ?, ?, ?, ?)`  
  ).run(patientName, medicationName, dosage, route,  
    scheduledAt, notes ?? null);  
  // ... buscar o registro criado e responder 201  
});
```

NOTES É OPCIONAL

isBlank(notes) ? null : notes.trim()

String vazia não é a mesma coisa que ausência de dado.
Trate notes como null quando vier vazio.

DEPOIS DO INSERT

Responda com o registro criado

Busque de volta pelo lastInsertRowid e devolva 201 — o cliente precisa saber o id gerado.

O formulário, ponta a ponta

APP.JS — CLIQUE EM CADASTRAR

```
saveButton.addEventListener('click', async () => {
  saveButton.disabled = true;

  try {
    const created = await createMedication({
      patientName: patientNameInput.value,
      medicationName: medicationNameInput.value,
      dosage: dosageInput.value,
      route: routeInput.value,
      scheduledAt: scheduledAtInput.value,
      notes: notesInput.value,
    });

    addMedication(created); // acao no estado
  } catch (error) {
    setFormFeedback(error.message, 'error');
  } finally {
    saveButton.disabled = false;
  }
});
```

POR QUE DESABILITAR O BOTÃO

POST não é idempotente

Duplo clique = duas prescrições iguais. Desabilitar durante o envio evita isso.

ADDMEDICATION

Você escreve isso em state.js

Acrescenta ao array: [...state.medications, medication].
Imutável, igual à Semana 01.

Antes de seguir para o Passo 4

- 1 Cadastrar pelo formulário grava no banco (confira reiniciando o servidor)
- 2 A lista atualiza sem recarregar a página
- 3 Deixar um campo obrigatório vazio mostra o erro do backend, não trava a tela
- 4 O botão fica desabilitado durante o envio

Metade do caminho. Os próximos dois passos são mais curtos — o padrão já está estabelecido.

53–67 MIN

Passo 4 — Obter um

GET /api/medications/:id: path param, 404 de verdade, painel de detalhe.



GET /api/medications/:id

SELECT COM WHERE ID = ?

```
app.get('/api/medications/:id', (request, response) => {  
  const row = db  
    .prepare(  
      `SELECT id, patient_name, medication_name, dosage,  
        route, scheduled_at, notes  
        FROM medication_orders WHERE id = ?`  
    )  
    .get(request.params.id);  
  
  if (!row) {  
    response.status(404).json({ error: 'Prescrição não encontrada.' });  
    return;  
  }  
  
  response.status(200).json(toMedicationJson(row));  
});
```

.GET() DEVOLVE UNDEFINED

Não lança erro sozinho

É você quem decide, com o if, que a ausência vira 404. O banco só informa: não achei.

REQUEST.PARAMS.ID

Path param, não query

:id na rota vira request.params.id no handler. Vem sempre como texto.

Clicar no cartão abre o detalhe

APP.JS — DELEGAÇÃO DE EVENTO NA LISTA

```
medicationListElement.addEventListener('click', (event) => {  
  const card = event.target.closest('[data-medication-id]');  
  if (card) openDetail(Number(card.dataset.medicationId));  
});  
  
async function openDetail(id) {  
  selectMedication(id); // acao no estado  
  try {  
    await getMedication(id); // pratica o path param + 404  
  } catch (error) {  
    setDetailError(error.message);  
  }  
}
```

POR QUE DELEGAÇÃO

Um listener, não um por cartão

A lista é redesenhada inteira a cada render. Um listener por cartão duplicaria a cada atualização.

RENDERDETAIL

Você escreve isso em render.js

Sem selectedMedication: esconda o painel. Com ele: desenha os campos + botão “Suspende”.

Antes de seguir para o Passo 5

- 1 Clicar num cartão abre o painel com os dados completos
- 2 O painel busca o dado por uma NOVA chamada à API — não reaproveita a lista
- 3 “Fechar” esconde o painel corretamente
- 4 Simular um id inexistente mostra erro tratado, não tela quebrada

Falta o último passo — e ele é o mais curto de todos.

67–80 MIN

Passo 5 — Remover

DELETE /api/medications/:id: o status 204, e suspender de verdade.



DELETE /api/medications/:id

O PASSO MAIS CURTO DO DIA

```
app.delete('/api/medications/:id', (request, response) => {  
  const existing = db  
    .prepare('SELECT id FROM medication_orders WHERE id = ?')  
    .get(request.params.id);  
  
  if (!existing) {  
    response.status(404).json({ error: 'Prescrição não encontrada.' });  
    return;  
  }  
  
  db.prepare('DELETE FROM medication_orders WHERE id = ?')  
    .run(request.params.id);  
  
  response.status(204).send(); // sem corpo!  
});
```

204 NÃO TEM CORPO

.send(), nunca .json()

Chamar `.json({})` aqui seria inconsistente com o próprio significado do status. Sem corpo é a resposta correta.

AINDA CHECA 404

Remover algo que já sumiu

Sim — validar antes de deletar continua sendo o padrão certo.

O botão “Suspend”

APP.JS — DELEGAÇÃO NO PAINEL DE DETALHE

```
detailPanelElement.addEventListener('click', async (event) => {  
  if (event.target.id === 'remove-button') {  
    const state = getState();  
    try {  
      await removeMedication(state.selectedId);  
      removeMedicationFromState(state.selectedId);  
    } catch (error) {  
      setDetailError(error.message);  
    }  
  }  
});
```

API.JS — REMOVEMEDICATION

204 não tem .json()

Não chame `response.json()` na resposta do DELETE — não há corpo para ler.

REMOVEMEDICATIONFROMSTATE

Filtra e fecha o painel

Tira o item do array e zera `selectedId` — a mesma ação faz as duas coisas.

O CRUD completo, testado ponta a ponta

- 1 Suspender remove da lista e fecha o painel de detalhe
- 2 O servidor responde 204, sem corpo
- 3 Remover duas vezes o mesmo id: a segunda vez dá 404
- 4 requests.http rodado do início ao fim, sem surpresas

As quatro operações estão de pé. O que falta agora é confirmar tudo junto — próximo slide.

Os oito itens, todos juntos

- 1 GET /api/medications devolve os registros do banco
- 2 A lista aparece na tela, vinda da API
- 3 Cadastrar pelo formulário grava no banco e atualiza a lista
- 4 Clicar num cartão mostra o detalhe, buscado por nova chamada
- 5 GET .../9999 devolve 404, e a tela trata esse caso
- 6 Suspend remove do banco, atualiza a lista, servidor responde 204
- 7 Nenhum fetch fora de api.js
- 8 Reiniciar o servidor (Ctrl+C + npm run dev): os dados continuam lá

Os erros mais prováveis do dia

Antes de chamar o professor, confira esta lista — resolve a maioria dos travamentos.



Campos chegam undefined no frontend → esqueceu de traduzir snake_case → camelCase



DELETE devolvendo erro estranho no frontend → tentou ler .json() de uma resposta 204



Clique no cartão não faz nada → o listener está dentro do map (deveria ser delegação no)



req.body vem vazio → faltou express.json() ou Content-Type no POST



Lista não atualiza depois de cadastrar → esqueceu de chamar addMedication() com o retorno do POST



no such table → esqueceu de rodar npm run db:reset

Para quem terminar antes dos 90 minutos

Nenhum dos dois é obrigatório. Reforçam conceitos já vistos, sem abrir escopo novo.

BÔNUS 1

Filtro por paciente

GET /api/medications?patientName=Ana filtrando no SQL (WHERE patient_name LIKE ?).

Pratica query string — Encontro 2 da Semana 01 — aplicada a uma consulta real.

BÔNUS 2

Ordenação justificada

Já está por padrão em ORDER BY scheduled_at ASC. Escreva uma linha no README explicando por que ordenar no SQL, e não no JavaScript.

Uma pergunta só

Qual das quatro operações vocês entenderam melhor de primeira, e qual custou mais?

E TAMBÉM

Uma linha sobre IA

Usaram? Para quê? Alguma sugestão que vocês recusaram? Não precisa ser longo — uma frase honesta já serve.

Sem nota. Isso vira insumo de diagnóstico para a próxima semana.

O QUE VOCÊS ACABARAM DE PROVAR

Quatro frases

- 1 O padrão da Semana 01 não era sorte — é replicável.
- 2 SQL parametrizado nas quatro rotas, sem exceção.
- 3 Todo fetch morou em api.js — de novo.
- 4 Uma IA em modo chat ajuda mais quando você já sabe o que perguntar.

Bom trabalho — entreguem quando o checklist do slide 33 estiver completo.